

H7. 회귀로 예측

naive의 왕좌에 도전할 선수를 우리 손으로 만든다 — 재료를 저울에 달아 가장 좋은 배합을 찾는 도구, 회귀

지난 시간 복습 — H6

- 규칙(홀드아웃)과 심판(MAE·RMSE)을 세우고 정식 경기를 치렀다.
- 왕좌는 제일 게으른 **naive**(MAE 1.6도) — 매끈한 도구들은 다 졌다. 이유는 H5의 언어로: 시차 1 자기상관 0.98.
- 그리고 약점도 확인했다 — naive는 전환점에서 하루 늦는다.
- 오늘의 예고편이 그때 나왔다: "naive를 이기려면 변화를 미리 아는 재료가 필요하다."

오늘의 목표

- "직선은 예측기다"를 설명할 수 있다 — naive가 사실 어떤 직선인지 포함해서.
- 회귀가 무엇을 찾아 주는지 말할 수 있다: 심판 앞에서 최강인 기울기와 절편.
- 특징 테이블을 만들고 scikit-learn 세 동작(fit / 계수 읽기 / predict)을 쓸 수 있다.
- 재료를 늘려 가며 어떤 재료가 살고 죽는지 실험으로 가릴 수 있다 — 그리고 승부의 결과를 정직하게 읽는다.

손으로 워밍업 — 직선은 예측기다

- H5의 lag plot: 가로축 어제, 세로축 오늘, 점 1,095개가 대각선 띠에 붙어 있었다.
- 이 그림에 직선을 하나 그으면 그게 곧 **예측기다** — 어제 기온에서 직선 위로 올라가면 오늘의 예측.
- 사실 naive도 직선이다: **예측 = 어제**, 기울기 1, 절편 0인 $y = x$.
- 그럼 다른 직선은? "어제와 3년 평균의 중간"을 말하는 신중한 직선 **$0.5 \times \text{어제} + 6.4$** 를 한파 주간에 적용해 보자 — 결과는 활동지에서.

좋은 직선의 기준 — 회귀

- 직선은 기울기와 절편, 숫자 두 개로 정해진다 — 그 두 숫자가 성적을 좌우한다(신중 직선의 참패가 증거).
- "가장 좋은 두 숫자"를 고르는 기준은 이미 있다 — H6의 심판.
- 훈련 구간에서 오차 제공의 합이 가장 작아지는 직선을 찾는 일이 회귀다. RMSE 심판 앞에서 최강인 직선.
- '회귀'라는 이름은 하는 일과 무관한 역사적 유물이다 — 유래는 첫 선수의 계수에서 눈으로 만난다.
- 이 찾기는 컴퓨터가 해 준다 — 우리가 정하는 것은 단 하나, 무엇을 재료로 줄 것인가.

특징 테이블 — 회귀는 표로 배운다

재료(특징) 열과 정답 열을 날짜마다 한 줄에 놓는다:

```
pd.DataFrame({"어제": t.shift(1), "오늘": t})
```

= "어제" 열과 "오늘" 열을 나란히 놓은 표

```
.dropna()
```

= NaN이 든 줄은 버려라 — 첫날은 어제가 없다

```
data = pd.DataFrame({"어제": t.shift(1), "오늘": t}).dropna()  
train = data[:"2025-07-31"]  
test = data["2025-08-01":]
```

- 분할선은 H6 그대로 — 훈련 일수가 H6과 하루 다른데, 어디로 갔는지는 활동지에서.

scikit-learn 읽는 법 — 배워라

```
model1 = LinearRegression()
```

= 새 회귀 모델을 준비하라 (직선이라서 Linear)

```
model1.fit(train[["어제"]], train["오늘"])
```

= 훈련 구간의 (재료, 정답)으로 배워라 — 오차 제곱 합이 가장 작은 직선을 찾는다

(`[["어제"]]` 두 겹 대괄호 = 열 목록으로 골라 표 모양 유지 — 회귀는 재료를 언제나 표로 받는다)

- 배우고 나면 `coef_` (재료마다의 기울기, 계수)와 `intercept_` (절편)가 생긴다 — 끝의 밑줄은 "학습 뒤에 생긴 값"이라는 표기 버릇.
- `coef_` 는 `array([...])` 목록으로 나온다 — 재료마다 계수가 하나씩이라서. 고장이 아니다.

첫 회귀 선수 — 어제 하나로

```
from sklearn.linear_model import LinearRegression

model1 = LinearRegression()
model1.fit(train[["어제"]], train["오늘"])
model1.coef_, model1.intercept_
```

- 성적보다 이 두 숫자를 먼저 본다 — 성적은 **얼마나** 잘하는지, 계수와 절편은 **어떻게** 예측하는지를 말해 준다: 기울기 = 재료를 얼마나 믿나, 절편 = 심어 둔 밑값.
- 그 기울기 — 어디서 많이 본 숫자다(H5). 그리고 이 직선은 1절의 두 직선 중 하나와 거의 같다. 확인은 활동지에서.
- 이름의 유래: 갈턴이 아버지와 아들의 키에서 발견한 "극단은 평균으로 되돌아온다(regress)" — 우리 직선의 기울기가 1보다 살짝 작은 이유와 같은 이야기다.

predict 읽는 법 — 답안을 써라

```
model1.predict(test[["어제"]])
```

= 시험 구간의 재료로 답안을 써라

```
pd.Series(..., index=test.index)
```

= predict는 날짜 없는 숫자 뭉치를 돌려주므로 시험 구간의 날짜를 도로 입힌다 — 그래야 채점이 된다

```
pred1 = pd.Series(model1.predict(test[["어제"]]), index=test.index)
e1 = test["오늘"] - pred1
e1.abs().mean(), (e1**2).mean() ** 0.5
```

- naive의 성적은 같은 표에서 바로 나온다 — "어제" 열이 곧 naive의 예측이므로 `test["오늘"] - test["어제"]`.

첫 승부 — 그리고 오늘의 주제문

- 회귀(어제) vs naive — 이번엔 소수 둘째 자리까지 읽는다.
결과는 활동지에서 확인하지만, 미리 말하면: 놀랍도록 가깝다.
- 회귀는 훈련 2년에서 가장 좋은 직선을 찾았는데도 왜 이 정도일까 — 재료는 어제 숫자 하나뿐이고, naive가 그 숫자를 그대로 내며 이미 다 쓰고 있다. 같은 재료로는 같은 답.
- 오늘의 주제문: 성적은 방법이 아니라 재료가 정한다.
- 그러니 이기려면 새 재료가 필요하다 — H4에서 만든 것 중에 후보가 있다.

새 재료 — 달의 보통

- H4의 월별 프로파일("그 달의 보통")을 재료로 데려온다. 단, H6의 규칙대로 훈련 구간에서만 계산한다:

```
train_t = t[:"2025-07-31"]  
profile = train_t.groupby(train_t.index.month).mean()
```

- 프로파일은 **12줄** — 7/31은 끝점일 뿐, 데이터 시작이 2023년 8월이라 훈련 2년에 열두 달이 모두(두 번씩) 들어 있다.
- 이 12줄을 1,096일에 입히는 도구가 **map**이다:

```
month.map(profile)
```

= 월 번호를 이름표 삼아 profile 표에서 값을 찾아와라 —

8월 10일 줄에는 8월의 보통이 온다 (H4 transform과 같은 일을 "훈련에서 배운 표"로)

재료 둘의 회귀 — 레시피를 읽는다

```
model2 = LinearRegression()  
model2.fit(train2[["어제", "달의보통"]], train2["오늘"])  
model2.coef_, model2.intercept_
```

- 이제 계수가 두 개 나온다 — 재료마다 하나씩, 칸의 순서는 **fit에 준 열 순서 그대로**(첫 칸 어제, 둘째 칸 달의 보통).
회귀가 찾은 것은 직선이 아니라 **배합 비율**이다.
- H3에서 지수평활을 "새 소식 30% + 기억 70%"로 읽었듯, 이 배합도 문장으로 읽는다: "오늘의 예측 = 어제 ____% + 달의 보통 ____%" — 비율은 활동지에서.
- 채점 결과는 두 심판이 **다르게** 알아본다 — 누가 크게 알아보는지, 그 이유가 H6 1절과 이어진다.

어디서 이겼나 — 정직하게 읽기

- MAE 차이는 소수 둘째 자리 — 이것만으로 우승 선언은 금물 (H6: "둘째 자리로 우열을 가리지 말라").
- 개선이 진짜인지는 어디서 벌었는지 — 어느 달에서 naive보다 오차를 줄였는지 — 로 판단한다: 오차를 월별로 쪼개 naive와 나란히 놓는다.
- 회귀가 크게 번 달들은 어떤 계절인가 — H6에서 naive의 아픈 달이라 부른 그 계절인가? 손해 본 달은 없나?
- 그리고 naive 최악의 날(2026-02-06)에 회귀는 몇 도를 말했나 — 레시피로 손 계산해 본다. 전환점의 숙제는 풀렸을까?

자료 실험 — 무엇이 살고 무엇이 죽나

- 작년 같은 날(H6의 계절 naive를 자료로): 회귀는 이 자료를 쓸까? 계수를 보면 회귀의 판단이 보인다 — 그리고 이 자료는 공짜가 아니다(`len(train3)` 을 볼 것).
- 요일의 보통: 프로필을 먼저 보라 —

```
week = train_t.groupby(train_t.index.dayofweek).mean()
```

- 자료의 정보량은 프로필의 굴곡이다. 넣기 전에 성적을 예언할 수 있다 — 예언과 검증은 활동지에서.
- 자료를 가리는 눈: 죽은 자료(정보 없음), 겹치는 자료(이미 아는 것), 새는 자료(미래) — 세 번째가 가장 위험하다.

오늘 배운 세 문장

1. 회귀는 심판 앞에서 최강인 배합을 찾아 준다 — 직선의 두 숫자든, 재료 여럿의 비율이든.
2. 성적은 방법이 아니라 재료가 정한다 — 죽은 재료, 겹치는 재료, 새는 재료를 가리는 눈이 실력이다.
3. naive는 거의 못 넘는 벽이었다 — 큰 오차를 깬 것이 진짜 소득이고, 전환점은 여전히 숙제다.

이제 활동지로

1. 신중한 직선을 한파 주간에 적용해 참패를 확인하고, "좋은 직선의 기준"을 세운다.
2. 특징 테이블을 만들어 첫 회귀를 배우게 하고, 찾아낸 기울기의 정체(H5에서 본 숫자)를 밝힌다.
3. 달의 보통을 재료로 추가해 레시피를 읽고, 두 심판의 반응 차이를 해석한다.
4. 월별 분해와 2026-02-06 손 계산으로 "어디서 이겼나"를 정직하게 읽는다.
5. 작년과 요일 재료를 실험해 재료 가리는 눈을 기른다 — 그리고 분식집이라면 무엇이 살아나는지까지.